COS30017 Assignment 2 Report – Fake Uber

Nov 8, 2026 · @bchung

1. Introduction

Fake Uber is a working Android proof of concept for weekly car rentals: it shows five cars one at a time, lets a user rent one for 1–7 days with a 500-credit balance, and passes 10 out of 10 Espresso UI tests.

The brief asked for a simple car-rental app for a company expanding into weekly rentals of used and new cars. My tutor's guidance was that the logic should stay simple and that UI/UX is the priority, so most of the design time went into research, sketching and accessibility before any code was written.

Item	Detail
App name	Fake Uber (package `com.example.fakeuber`)
Language and UI toolkit	Kotlin, XML Views with ViewBinding
Screens	2 Activities: `MainActivity` (browse) and `RentActivity` (rent details)
Target	compileSdk / targetSdk 37, minSdk 24 (Android 7.0, about 99.5% of devices)
Data	5 cars held in memory only, no disk storage
Currency	Credits (1 credit = 10,000 VND)
Tests	10 Espresso UI tests, all passing in 43 s

The app covers every requirement in the brief: one car at a time with a Next button, details under the image, renting through a Parcelable passed in an Intent, a credit balance with a 400-credit limit per rental, Save and Back behaviour, Snackbar feedback, favourites, search, sort, a dark-mode switch on both screens, at least two non-TextView widgets, reusable styles and error checking.

2. User stories and use cases

Two user stories cover the app's two core needs, choosing a suitable car and renting it without overspending; each has a matching use case that walks through every step, including the error branches. Every acceptance criterion below is checked by an Espresso test or a manual test in section 8.

US1: Browse, find and compare cars

> *As a customer looking for a short rental, I want to see each car's price, rating, year and kilometres under its photo, and to search, sort and save cars, so that I can pick the right one before I start a booking.*

Acceptance criteria:

1. Home shows one car at a time with brand, model, year, rating, kilometres and price per day under the photo, plus a "1 / 5" position counter.
2. Next moves to the next available car and wraps around; rented cars are skipped.
3. Typing in the search field keeps only cars whose brand or model matches, ignoring case. No match shows "No cars match “xyz”" with a Clear search button.
4. The Sort menu offers Rating (high to low), Year (newest first) and Cost (lowest first), ticks the active option and reorders a copy of the list, never the base data.
5. Tapping the heart or long-pressing the photo adds the car to Favourites; tapping again removes it.

US2: Rent within my credit, with my documents

> *As a customer with 500 credits, I want to see the total cost and my balance after rental before I confirm, and to add my ID and licence photos in the same place, so that I never book a car I cannot afford or am not allowed to drive.*

Acceptance criteria:

1. The credit balance is visible in the header of Home and Rent.
2. On Rent, the total (price × days), the balance after rental and the 400-credit limit bar update as soon as the 1–7 day slider moves.
3. Save is refused, with the reason shown in place, if the total is over 400, over the balance, or a document photo is missing. The user stays on Rent.
4. A valid Save returns to Home, deducts the total, removes the car from Available now, lists it under My rentals and shows "… booked" in a Snackbar.
5. Back saves nothing and shows "Rental not saved". Cancel booking refunds the credits and returns the car to the list.

UC1: Rent a car (implements US2)

Field	Description
Actor	Customer
Goal	Rent a car for 1–7 days without breaking the credit rules
Precondition	Home is open and at least one car is available
Success	Credits deducted, car listed under My rentals and removed from Available now
Cancel	Nothing changes; the balance stays the same

Main flow

1. The customer presses **Next** until the wanted car is shown, then presses **Rent**.
2. The app opens the Rent screen and passes the car in an explicit Intent as a Parcelable.
3. The app shows the photo, details, the slider at 1 day, the Credits card and two empty document tiles.
4. The customer moves the slider; the total, balance after rental and limit bar update at once.
5. The customer taps **Driver ID Photo** and **Driver Licence Photo** and picks a photo for each in the system Photo Picker. Each tile shows a tick.
6. The customer presses **Save · N credits**.
7. The app checks: both photos present, total ⩽ 400, total ⩽ balance.
8. The app returns to Home, deducts the credits, moves the car to My rentals and shows "Toyota Vios 1.5G booked · 3 days · 240 credits".

Alternative flows

- **7a. Total over 400:** the limit bar turns red and the Credits card says "This rental costs 440 credits. One rental can't go over 400…". The customer picks fewer days and returns to step 6.
- **7b. Total over balance:** the card says "Not enough credits. You have 170, this rental needs 180…". The customer picks fewer days or presses Back.
- **7c. A photo is missing:** the empty tile turns red with "Required" and a Snackbar says "Can't save yet". The customer adds the photo and returns to step 6.
- **3–7. Back pressed** (button, arrow or system back): the app returns to Home with no change and shows "Rental not saved".

- **After 8. Cancel booking:** the app refunds the credits, returns the car to the list and shows "Booking cancelled · N credits refunded".

UC2: Find, sort and save a favourite (implements US1)

Field	Description
Actor	Customer
Goal	Find a car by name or criterion and keep it for later
Precondition	Home is open
Success	The car appears in the Favourites row with a filled heart

Main flow

1. The customer types part of a name or model in the search field, for example "toyota".
2. The app filters as the customer types and the counter changes to "1 / 2".
3. The customer clears the search, opens **Sort** and picks "Cost · lowest first".
4. The app reorders a copy of the list and ticks the active option; the base list is unchanged.
5. The customer taps the heart on the photo.
6. The app fills the heart and adds a chip (thumbnail, model, price) to Favourites; the count reads "1 saved".

Alternative flows

- **2a. No match:** the app shows "No cars match “xyz”", the counter "0 / 0" and a Clear search button.
- **5a. Long-press on the photo:** same result as step 6, offered as a shortcut.
- **5b. Already a favourite:** tapping the heart removes it from Favourites.
- **After 6. Chip tapped:** the card jumps to that car; if it is rented, a Snackbar explains why it cannot be shown.

3. Two layout sketches and the chosen design

Layout A was chosen because every action is a visible control within thumb reach and it prevents credit mistakes instead of reporting them after Save. Both sketches are deliberately low-fidelity: grey boxes, crossed image placeholders, and red notes naming the Android widget behind each element.

Layout A: one scrolling card per screen. Home stacks titled blocks in one column: a header with the credit balance and a dark-mode Switch, an always-visible search field with a Sort button, a Favourites row, one car card with Rent and Next buttons, and My rentals. Rent repeats the car, then three cards in decision order: days (Slider), driver documents (two photo tiles) and credits (live total and limit bar), with a fixed Back/Save bar.

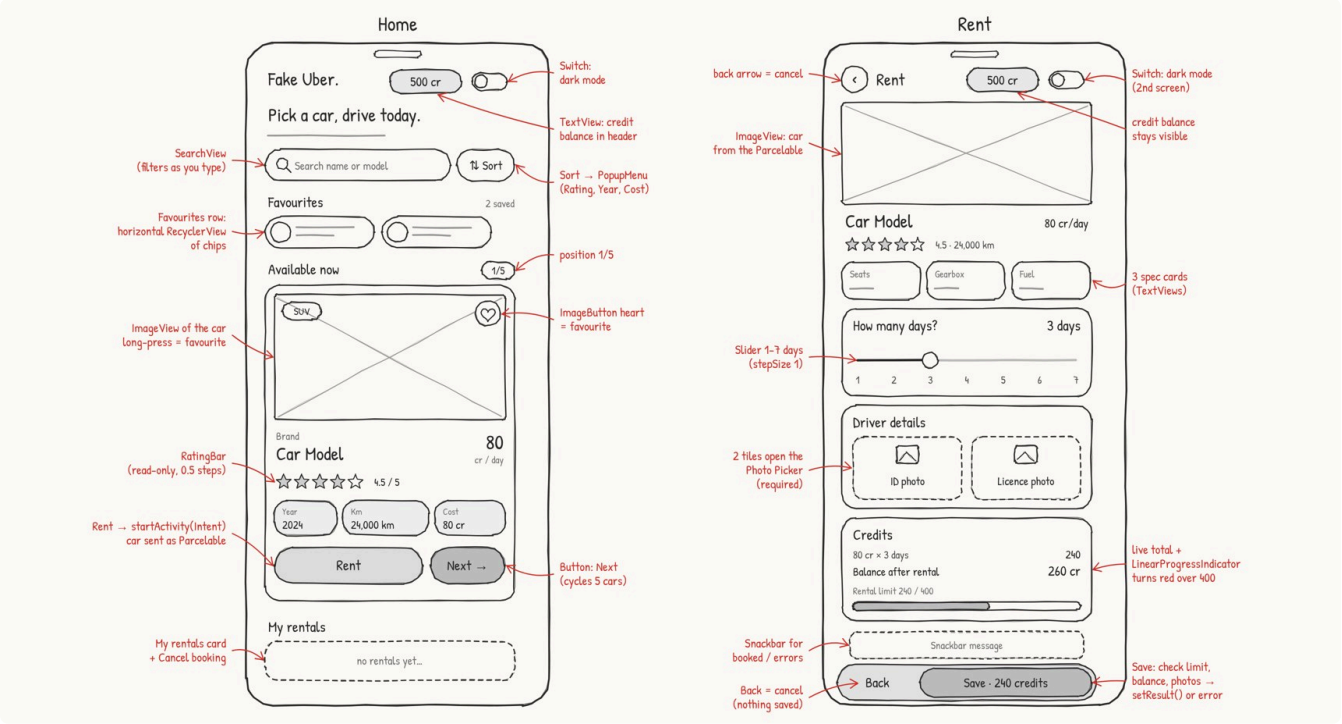


Figure 1. Layout A low-fi sketch (Home and Rent) with widget notes.

Layout B: app bar, swipe pager and table. A Toolbar holds the title, balance and Switch; cars are swiped in a ViewPager2 with dots; sorting is a ChipGroup; details sit in a TableLayout; favourites collapse into an ExpandableListView and bookings into a ListView. Rent uses a −/+ stepper instead of a slider, Upload buttons with CheckBoxes, a plain summary, an AlertDialog for errors, a Toast for confirmation and an Extended FAB to confirm.

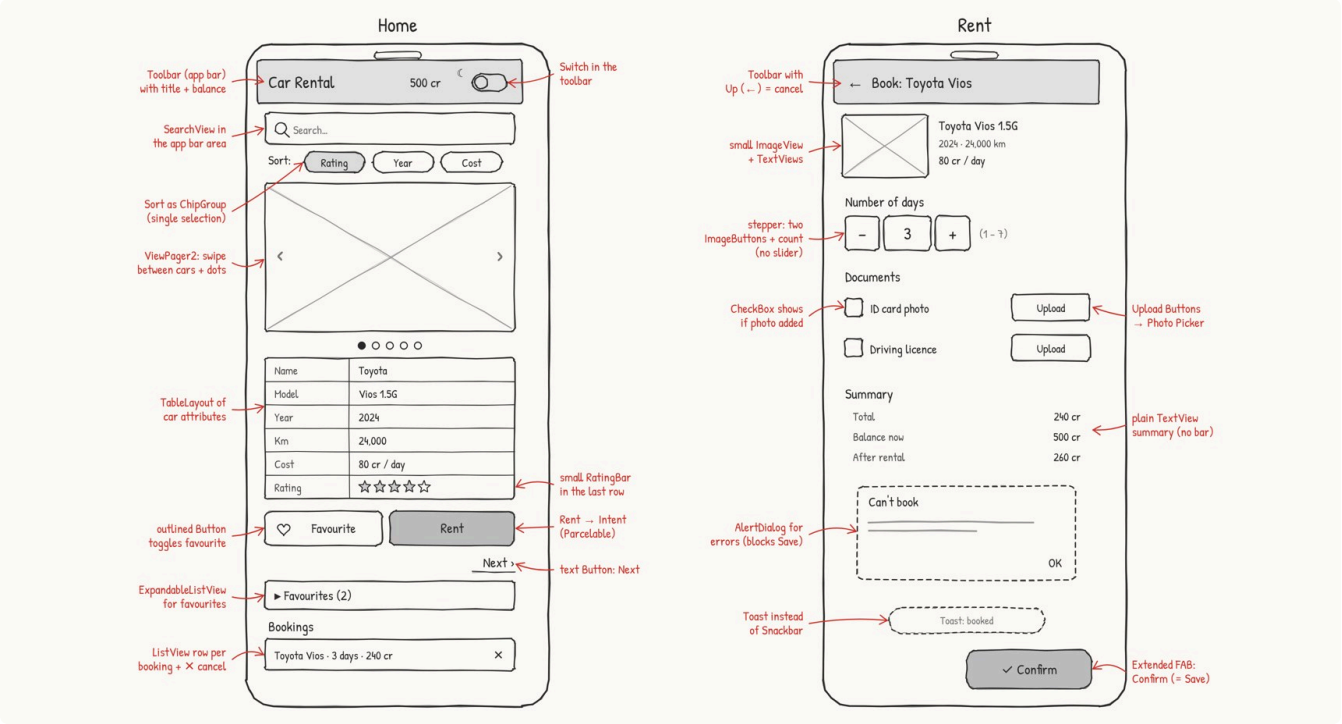


Figure 2. Layout B low-fi sketch (Home and Rent) with widget notes.

Element	Layout A	Layout B
Moving between cars	Next button + "1 / 5" counter	ViewPager2 swipe + dots + text Next
Sort	Button opening a PopupMenu	ChipGroup, always visible
Car details	Stat chips + RatingBar under the photo	TableLayout rows + small RatingBar
Favourites	Horizontal row of chips (RecyclerView)	ExpandableListView
Bookings	My rentals cards with Cancel booking	ListView row with ✕
Number of days	Slider 1–7	Stepper: two ImageButtons + count
Driver documents	Two large photo tiles	Upload Buttons + CheckBoxes
Cost	Credits card with live limit bar	Plain TextView summary
Errors	Inline message + Snackbar	AlertDialog
Confirmation	Snackbar	Toast

Both layouts meet every requirement in the brief, including at least two non-TextView widgets (A: RatingBar, Slider, Switch; B: RatingBar, stepper, Switch). The choice came down to these criteria:

Criterion	Better	Why
Discoverability of moving between cars	A	A visible Next button and counter; a swipe is a hidden gesture and dots are weak cues (Budiu, 2018)
Preventing over-limit rentals	A	The live total and limit bar warn before Save (Nielsen heuristic 5, error prevention)
Quality of error messages	A	Errors appear in place with the exact numbers; a modal dialog blocks the screen (Neusesser & Sunwall, 2023)
Feedback that can be built as drawn	A	A Snackbar sits at the bottom and follows the theme; a text Toast can no longer be positioned on Android 11+
Choosing days	A	One drag across a visible 1–7 range instead of up to 6 taps on small −/+ targets (Parhi et al., 2006)
Espresso testing	A	Clicks and text checks; swipes and dialogs need extra handling
Taps to sort	B	1 tap on a chip instead of 2 (open menu, choose)
Information density	B	A table shows every attribute at once

B's advantages are small for a five-car app with three sort options, so Layout A became the final design. The final app follows Layout A closely; the one deliberate change is the third spec card on Rent, which shows the car **Type** instead of the sketched **Gearbox**, because all five cars are automatic and the card would have shown the same value every time.

4. UI/UX research and design choices

Every visual decision in the app traces back to a research finding, and the biggest change research forced was the colour palette: three colours in the first design failed WCAG contrast and were replaced before any code was written.

One decision per screen area. Home answers "which car?" and Rent answers "how long, and can I afford it?". Rent's cards follow the order of the decision (days, then documents, then credits), so the total is always visible next to the control that changes it.

Prevent errors instead of reporting them. The Credits card shows the total, the balance after rental and a bar measuring the 400-credit limit while the user is still choosing. Over-limit and not-enough-credit problems appear live as the slider moves, with the exact numbers and the fix ("Pick fewer days or press Back to cancel"). Missing photos are only flagged after Save, so the user is not warned before having a chance to add them.

Feedback the user can trust. Every outcome (booked, not saved, cancelled and refunded, can't save yet) shows a two-line Snackbar: a bold title and one line of detail. A Snackbar was chosen over a Toast because it follows the app theme, is read by TalkBack, and can sit just above the Back/Save bar (Material Components for Android, n.d.); a text Toast's `setGravity()` has no effect from Android 11 (Android Developers, n.d.-f).

Documents without permissions. The two document tiles open the system Photo Picker (`PickVisualMedia`), which needs no storage permission: the user shares only the one photo they choose (Android Developers, n.d.-e). A tick and a thumbnail confirm each photo.

Accessible colour. Colours were checked against WCAG 2.2 contrast minimums: 4.5:1 for normal text and 3:1 for large text and UI parts (W3C, 2023).

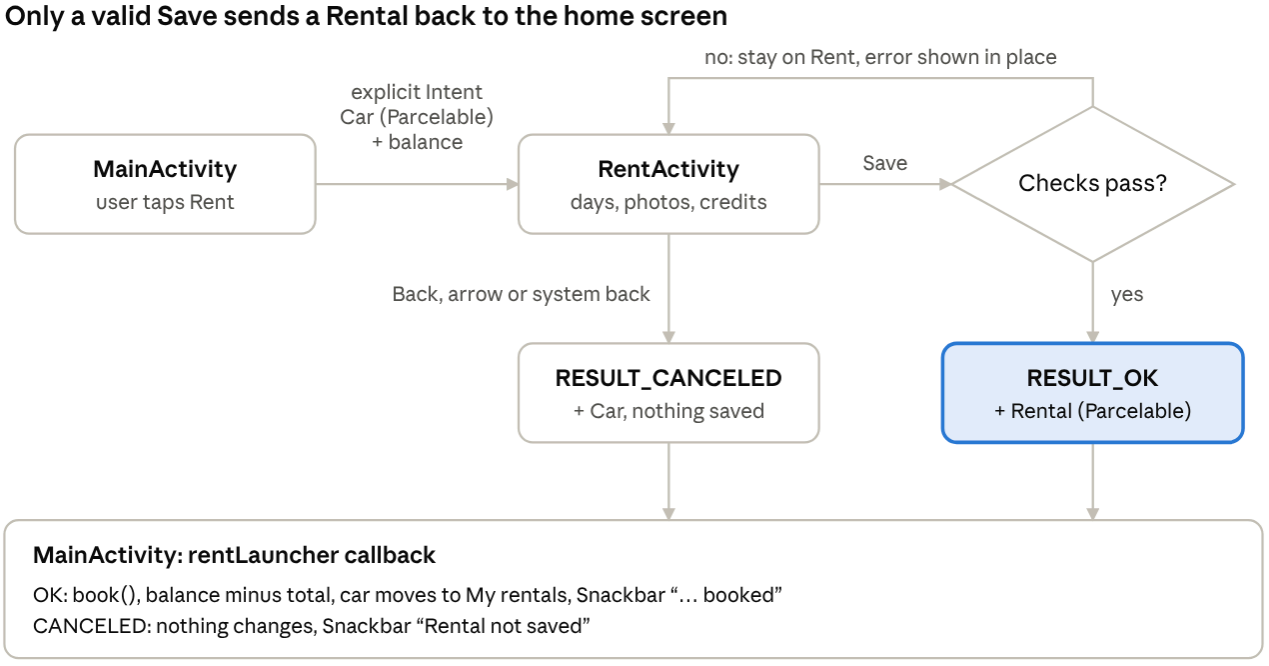
Colour role	First design	Final	Reason
Brand orange (buttons, slider, accents)	#F97316 with white text	#EA580C with **dark** text	White on orange was about 2.8:1; dark text on #EA580C reaches 4.9:1
Orange used as small text ("Clear search")	#F97316	#C2410C	5.2:1 on the card surface
Secondary grey text	#716C65	#625D57	5.8:1 on the page background (was under 4.5:1)
"Rented" green	#2F9E5E	#1F7A45 (light), #4ADE80 (dark)	5.2:1; the dark theme needs a brighter green
Error red	#E11D2E	#B91C1C (light), #F87171 (dark)	5.3:1 inside the pink error box; 6:1 on dark

Touch targets of at least 48dp. The heart, switch, back arrow, document tiles and every button are at least 48 × 48dp, the Android minimum, and the main buttons are 56dp tall for one-handed use (Parhi et al., 2006).

Screen reader support. The car photo, heart, document tiles and Cancel booking buttons have content descriptions that name the car or the action. Section titles are marked as headings, the error box is a live region so TalkBack reads changes, and the photo's long-press carries a custom TalkBack label ("add or remove favourite").

5. Implementation: architecture, Intent and Parcelable

The app is two Activities that talk only through Intents: `MainActivity` sends the chosen car to `RentActivity` as a Parcelable, and `RentActivity` sends back either a Parcelable `Rental` (Save) or a cancellation (Back). All session state lives in two ViewModels, so it survives the dark-mode switch while still being memory-only (Android Developers, n.d.-g).



Renting flow between the two Activities

The only way data comes back to Home is a valid Save; every Back path returns a cancellation and changes nothing.

Structure

File	Responsibility
`model/Car.kt`, `model/Rental.kt`	Data classes, each implementing `Parcelable` by hand
`model/CarRepository.kt`	The 5 cars as a read-only `List`, the 500 start balance and the 400 limit

File	Responsibility
`MainViewModel.kt`	Balance, rentals, favourites, search text, sort order, current index
`MainActivity.kt`	Draws Home from the ViewModel; launches Rent and handles its result
`RentViewModel.kt`	Days, the two photo Uris, credit and photo validation
`RentActivity.kt`	Draws Rent; Photo Picker; Save and Back
`ui/DarkModeToggle.kt`, `ui/FuSnackbar.kt`, `ui/FavouriteAdapter.kt`	Shared helpers used by both screens

Every screen follows one pattern: **user action → ViewModel function → `render()`**. `render()` redraws everything from the ViewModel, so the UI cannot drift out of sync with the data, and after the Activity is recreated (dark mode, rotation) one call restores the screen exactly.

The base data is never changed. `displayedCars()` builds a new list each time: cars not rented → matching the search → sorted with `sortedBy` / `sortedByDescending`. Next wraps with `(index + 1) % size`.

Components of the Intent

```
fun newIntent(context: Context, car: Car, balance: Int): Intent =
    Intent(context, RentActivity::class.java)   // explicit: names the exact
class
        .putExtra(EXTRA_CAR, car)                // Parcelable extra
        .putExtra(EXTRA_BALANCE, balance)        // primitive extra
```

Component	In Fake Uber	Purpose
Context	`MainActivity`	Where the Intent is sent from
Component (target class)	`RentActivity::class.java`	Makes it an **explicit** Intent: Android starts that class directly, with no search for a matching app, so no action, category or data URI is needed (Android Developers, n.d.-c)
Extras	`EXTRA_CAR` (Parcelable `Car`), `EXTRA_BALANCE` (`Int`)	The data the second screen needs, as key–value pairs in a Bundle

Component	In Fake Uber	Purpose
Launcher	`registerForActivityResult(StartActivityForResult())`	Starts Rent and receives its answer (Android Developers, n.d.-b) (replaces the deprecated `startActivityForResult`)
Result code	`RESULT_OK` / `RESULT_CANCELED`	Save versus Back
Result extras	`EXTRA_RENTAL` (Parcelable `Rental`) or `EXTRA_CAR`	The booking, or the car's name for the "not saved" message

On the receiving side, `IntentCompat.getParcelableExtra(intent, EXTRA_CAR, Car::class.java)` reads the car in a type-safe way that works on both old and new Android versions. Back has three entry points (the Back button, the arrow and the system back gesture through `OnBackPressedDispatcher`), and all three call the same `cancelRental()`, so no path can save by accident.

Why Parcelable

`Parcelable` is Android's own serialisation format for passing objects between components: the object writes its fields into a flat `Parcel`, and a `CREATOR` rebuilds it on the other side (Android Developers, n.d.-d).

```
override fun writeToParcel(parcel: Parcel, flags: Int) {
    parcel.writeInt(id)
    parcel.writeString(brand)
    parcel.writeString(model)
    parcel.writeInt(year)
    // … the other fields, always in the same order
}
```

Advantages over Java's `Serializable`:

- **Faster and lighter.** No reflection and far fewer temporary objects, which matters on the main thread and when an Intent crosses process boundaries through Binder.
- **Explicit.** I decide exactly what is written; nothing private is captured by accident.
- **Nests cleanly.** `Rental.writeToParcel()` first calls `car.writeToParcel()`, so a Parcelable carries another Parcelable.
- **Type-safe reads** with `IntentCompat.getParcelableExtra(…, Car::class.java)`.

The cost is boilerplate and one strict rule: fields must be read in exactly the order they were written, or values silently land in the wrong field. I wrote both classes by hand instead of using `@Parcelize`, partly to understand this mechanism and partly because

the `kotlin-parcelize` plugin's version must match the Kotlin bundled with Android Gradle Plugin 9.

6. Feature walkthrough

The finished app matches the Layout A design on every screen; the figures below are screenshots from the Pixel 7 emulator.

Browsing: one car at a time, favourites, sort

Home shows one car at a time with brand, model, a RatingBar, year, kilometres and daily cost under the photo, plus a "1 / 5" counter and a Next button that wraps around. The heart (or a long-press on the photo) adds the car to the Favourites row; tapping a chip jumps back to that car. Sort opens a PopupMenu with the three required orders.

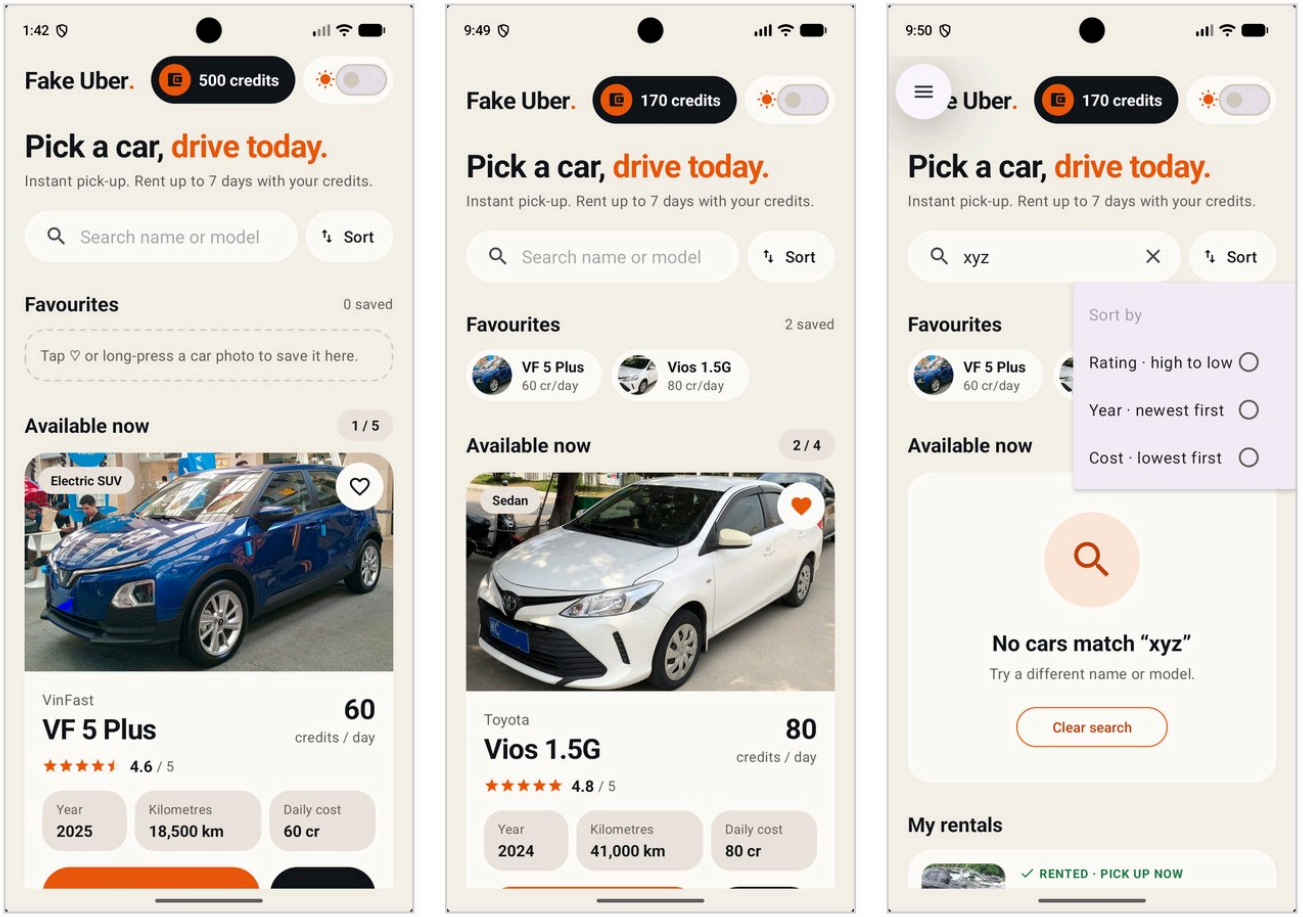


Figure 3. Home, Favourites with two saved cars, and the Sort menu.

Search

The SearchView filters by brand or model on every keystroke. When nothing matches, the card is replaced by "No cars match “xyz”" with a Clear search button and the counter reads "0 / 0".

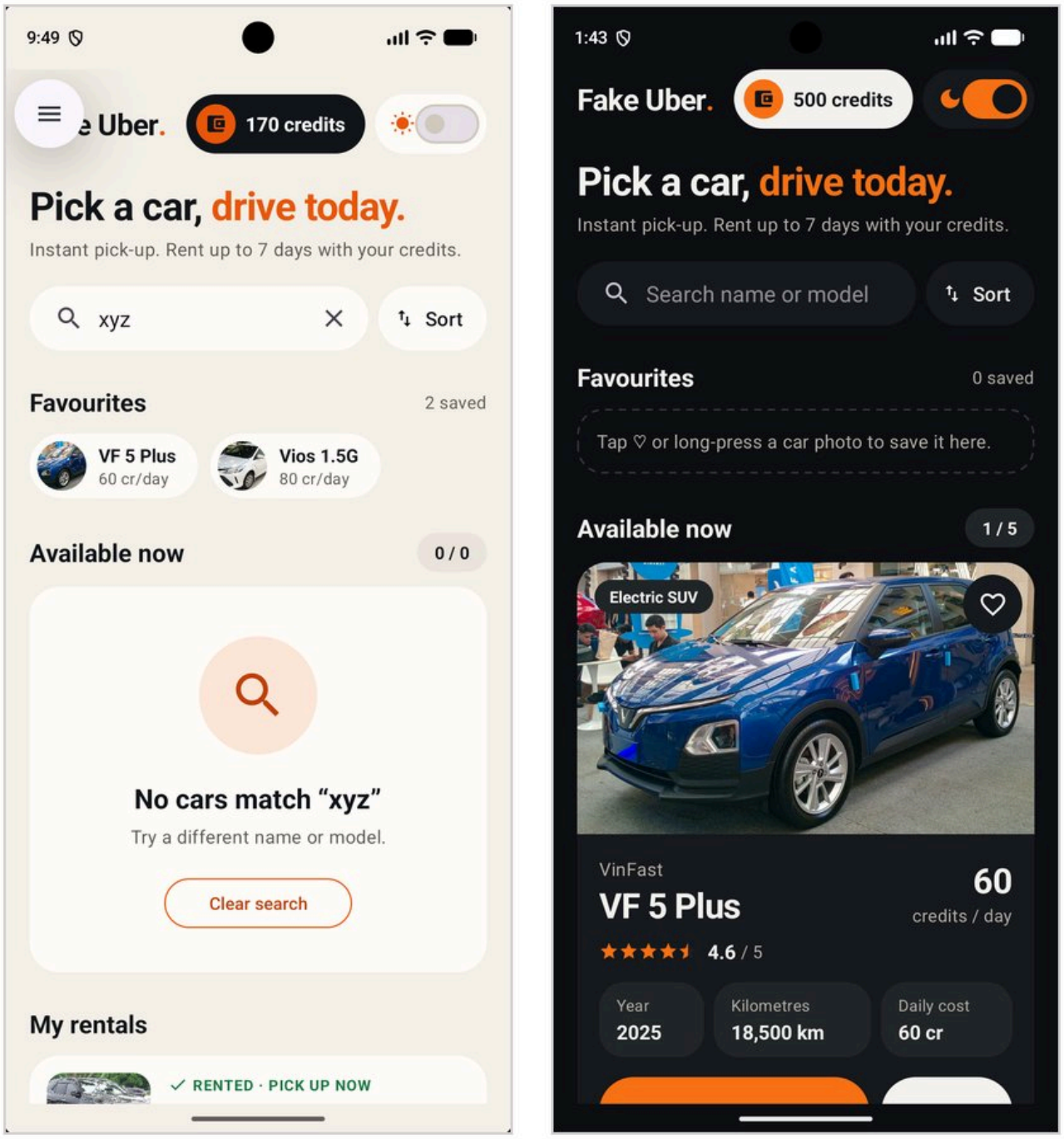


Figure 4. Search with no results, and Home in dark mode.

Renting

Rent shows the car again with Seats, Fuel and Type cards, a Slider for 1–7 days, two document tiles that open the Photo Picker, and the Credits card. After Save, Home shows a "… booked" Snackbar, the balance drops, the car leaves Available now and appears under My rentals with a Cancel booking button.

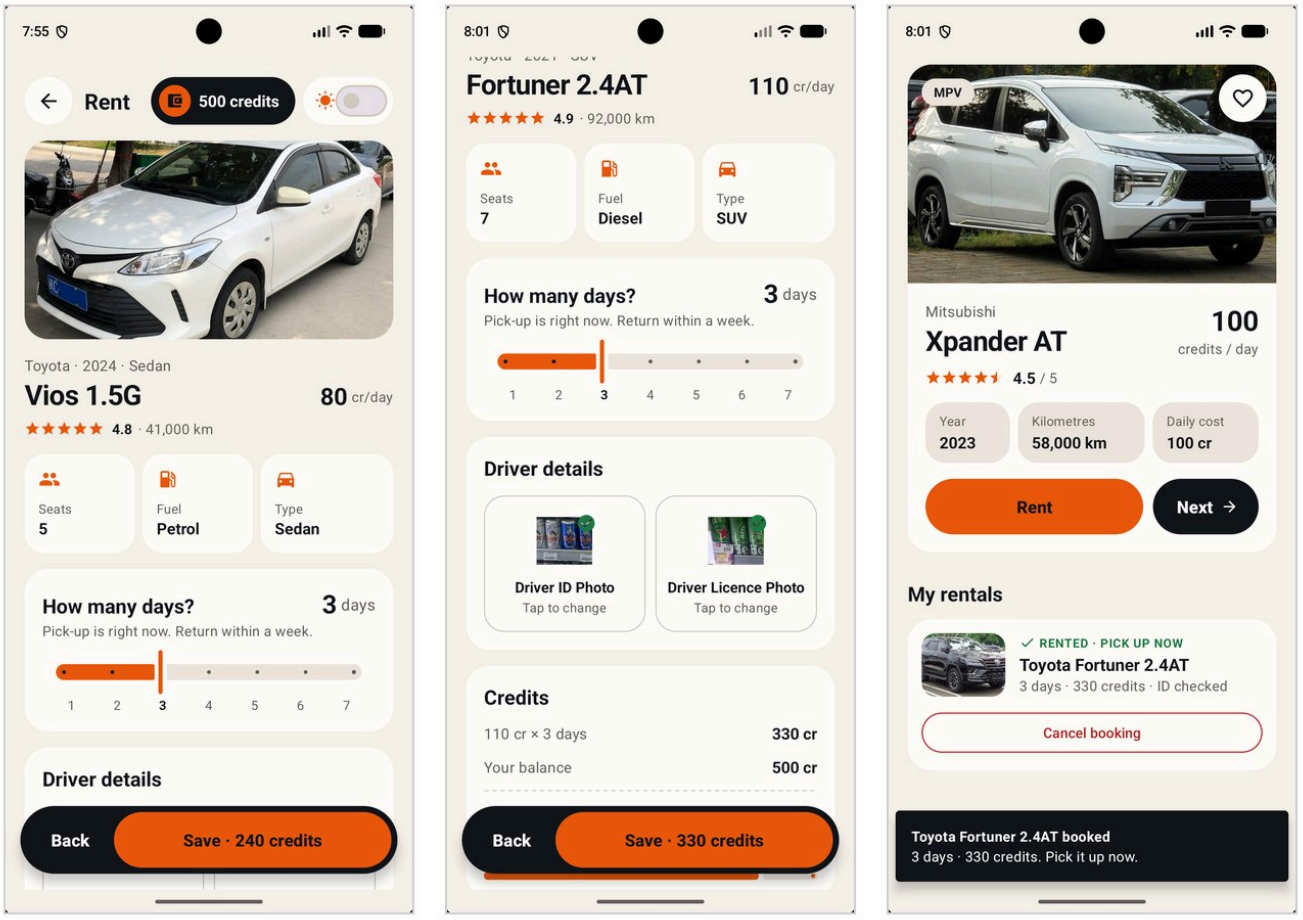


Figure 5. Rent screen, both document photos added, and the booking confirmed on Home.

Error checking

Three problems stop Save, and in every case the user stays on Rent until it is fixed or Back is pressed:

Problem	Example	What the user sees
Over the 400 limit	Fortuner × 4 days = 440	Red limit bar and "This rental costs 440 credits. One rental can't go over 400…"
Not enough credits	After a 330-credit booking, VF 5 × 3 days = 180 > 170	Balance after rental −10 cr in red and "Not enough credits. You have 170…"
Missing document photo	Save with no photos	Both tiles turn red with "Required" and a "Can't save yet" Snackbar

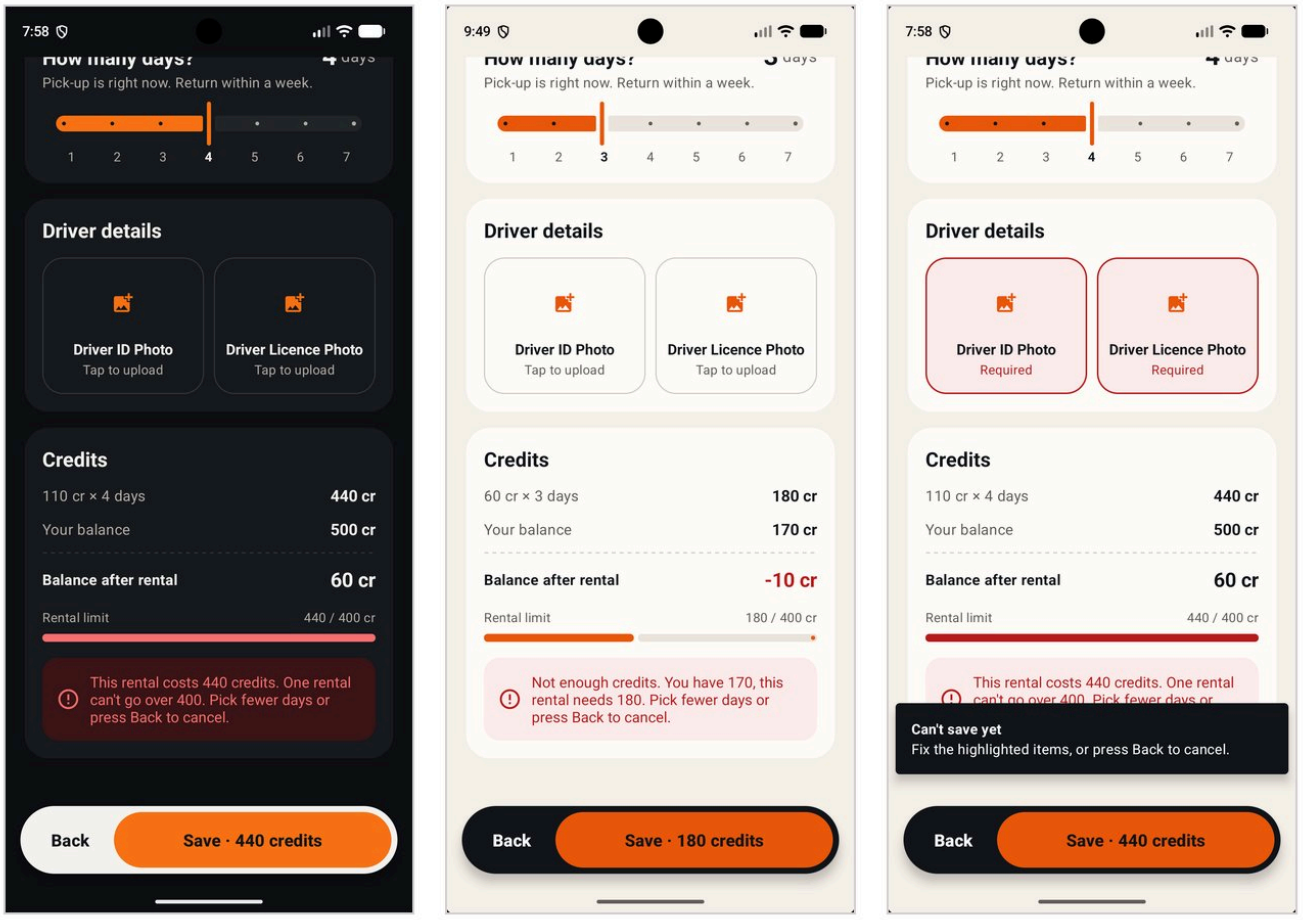


Figure 6. Over the limit, not enough credits, and missing photos.

Dark mode on both screens

The Switch in the header of Home and Rent changes the whole app. Every colour has a light and a dark value with the same name, so both screens restyle at once, and the chosen car, days and photos are kept.

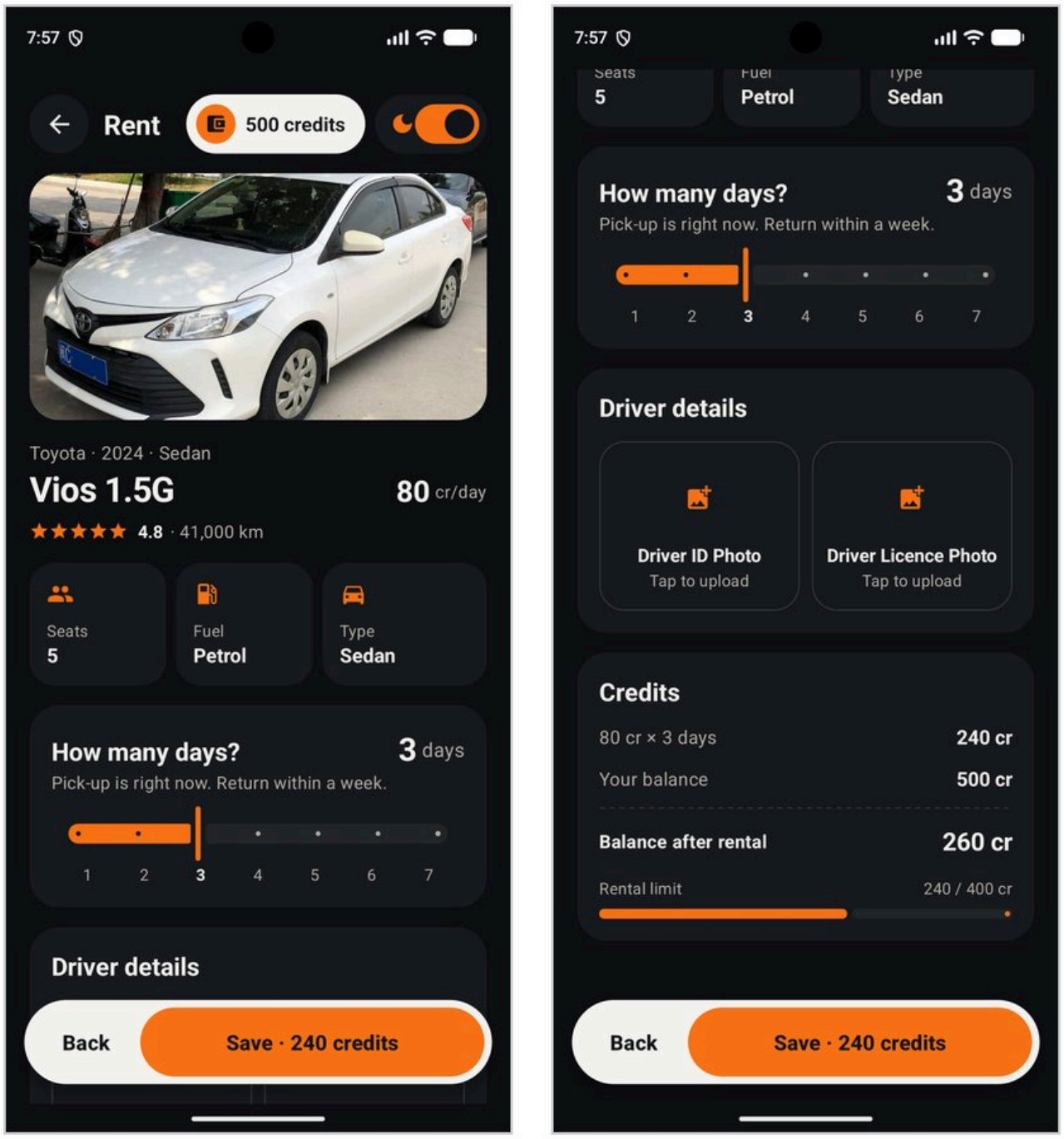


Figure 7. The Rent screen in dark mode.

Requirements coverage

Requirement from the brief	Where it is met
Five cars, data in memory, details under each image	`CarRepository`; car card on Home (Figure 3)
One car at a time with a Next button	Next wraps around; counter "1 / 5"

Requirement from the brief	Where it is met
Only free images	Wikimedia Commons, CC BY-SA 4.0 (section 11)
Rent opens a new screen with a Parcelable	Explicit Intent with `Car` (section 5)
Rented car leaves the list; cancelled car returns	My rentals + Cancel booking (Figure 5)
500 credits, ⩽ 400 per rental, balance in the header	Credit pill on both screens; live checks (Figure 6)
Save saves, Back cancels, result shown on the first screen	Activity Result API; Snackbars; My rentals
Toast or Snackbar when booked or not booked	Two-line Snackbar for every outcome
Favourites with a heart or long-press, separate section	Heart + long-press; Favourites row
Search bar and Sort menu (rating, year, cost) without changing base data	SearchView + PopupMenu on a copy of the list
Dark mode Switch, styles adapt on at least two screens	Switch on Home and Rent (Figures 4 and 7)
At least two non-TextView widgets	RatingBar, Slider (also Switch, LinearProgressIndicator)
Error checking that blocks returning until fixed or cancelled	`trySave()` returns null and the screen stays open
At least two styles used in at least two places	Six styles, each used on both screens (section 7)

7. Styles, themes, dark mode and accessibility

The app defines six reusable styles (plus one child style), and every one of them is used on both screens, which exceeds the brief's minimum of two styles in two places.

Style	Defines	Used on Home	Used on Rent
`TextAppearance.FakeUber.Title`	28sp bold, primary text colour	Logo, headline, car model	Car model

Style	Defines	Used on Home	Used on Rent
`TextAppearance.FakeUber.Section`	20sp bold	Favourites, Available now, My rentals	Rent title, How many days?, Driver details, Credits
`TextAppearance.FakeUber.Secondary`	14sp, secondary grey	Brand, "credits / day", stat labels, hints	Meta line, helper text, credit rows
`Widget.FakeUber.CreditPill`	Inverse pill, wallet badge, bold balance	Header	Header
`Widget.FakeUber.Button.Primary`	Orange fill, dark text, 56dp tall, pill shape	Rent	Save
`Widget.FakeUber.Button.Inverse`	Child of Primary; only the colours change	Next	Back
`Widget.FakeUber.Card`	Surface colour, 24dp corners, no elevation	Car card, empty-state card, rental cards	Photo, days, driver details and credits cards

One theme, two colour files. `Theme.FakeUber` extends `Theme.Material3.DayNight.NoActionBar` and maps the palette onto Material 3 colour roles (`colorPrimary` , `colorSurface` , `colorSecondaryContainer` , `colorSurfaceInverse` and others). Each colour exists twice with the same name, in `values/colors.xml` and `values-night/colors.xml` , so layouts only ever refer to `@color/fu_text` or `@color/fu_bg` and Android picks the right value. Because the roles are mapped, the Slider, Switch, progress bar and Snackbar restyle themselves without per-widget code.

The Switch. `DarkModeToggle.bind()` is shared by both screens. It reads the current mode from the configuration, sets the Switch first and only then attaches the listener, which calls `AppCompatDelegate.setDefaultNightMode()` . That call recreates every open Activity, so all state lives in the ViewModels. The Switch has `saveEnabled="false"` and

the listener ignores values equal to the current mode: without both guards, the Home and Rent screens restored each other's stale Switch state and flipped the app between light and dark forever (section 9, challenge 2).

Accessibility checklist

- Text contrast of at least 4.5:1 in both themes (section 4)
- Every tappable element at least 48 × 48dp
- Content descriptions on the car photo, heart, document tiles and Cancel booking
- Headings marked with `accessibilityHeading` for TalkBack navigation
- Error box as a polite live region
- Custom TalkBack label for the long-press shortcut
- Feedback at the bottom of the screen in a Snackbar

8. Testing

All 10 Espresso UI tests pass in 43 seconds on a Pixel 7 emulator, and a full manual run of every feature in both themes found no failures after the fixes listed in section 9.

The tests follow the brief's advice to test text and buttons rather than the RatingBar, and to remove redundancy. Every Espresso call lives in one **robot** class, `FakeUberRobot`, so each test reads like the user's steps:

```
@Test
fun notEnoughCredits_blocksSave() {
    with(robot) {
        next(4)
        bookCurrentCar(days = 3)   // Fortuner 330 → 170 left
        checkCredits(170)
        next()                     // wraps to VF 5
        rent()
        setDays(3)                 // 60 × 3 = 180 > 170
        checkTextContains(R.id.tvError, "Not enough credits")
    }
}
```

Test	What it proves
`next_movesThroughCarsAndWrapsAround`	Next goes 1/5 → 2/5 … and wraps back to 1/5
`save_booksCar_spendsCredits_andListsIt`	VF 5 × 3 days: balance 320, "1 / 4", Cancel booking shown

Test	What it proves
`back_cancels_andNothingChanges`	"Rental not saved", balance still 500
`overLimit_blocksSave`	Fortuner × 4 = 440 shows the limit error and stays on Rent
`notEnoughCredits_blocksSave`	180 > 170 shows "Not enough credits"
`missingPhotos_markTilesRequired`	Both tiles show "Required"
`cancelBooking_refundsCredits_andCarComesBack`	120 credits refunded, back to 500 and "1 / 5"
`search_filtersByName_andShowsEmptyState`	"toyota" → 1 / 2, "xyz" → 0 / 0, Clear search → 1 / 5
`sort_reordersByRatingYearAndCost`	Fortuner first by rating, VF 5 by year and by cost
`heart_addsCarToFavourites`	"0 saved" → "1 saved", Favourites row shown

Three pieces of test infrastructure were needed:

- **Photo Picker stub.** `espresso-intents` answers the picker's Intent at once with an image from the app's own resources, so tests never open the real gallery (Android Developers, n.d.-a).
- **Two custom ViewActions.** Espresso's `scrollTo()` does not support `NestedScrollView`, and it has no action for the Material Slider, so the robot adds `nestedScrollTo()` and `setSliderValue()`.
- **A conditional wait for Snackbars.** Snackbars queue, so `checkSnackbar()` re-checks every 100 ms for up to 5 s instead of sleeping for a fixed time.

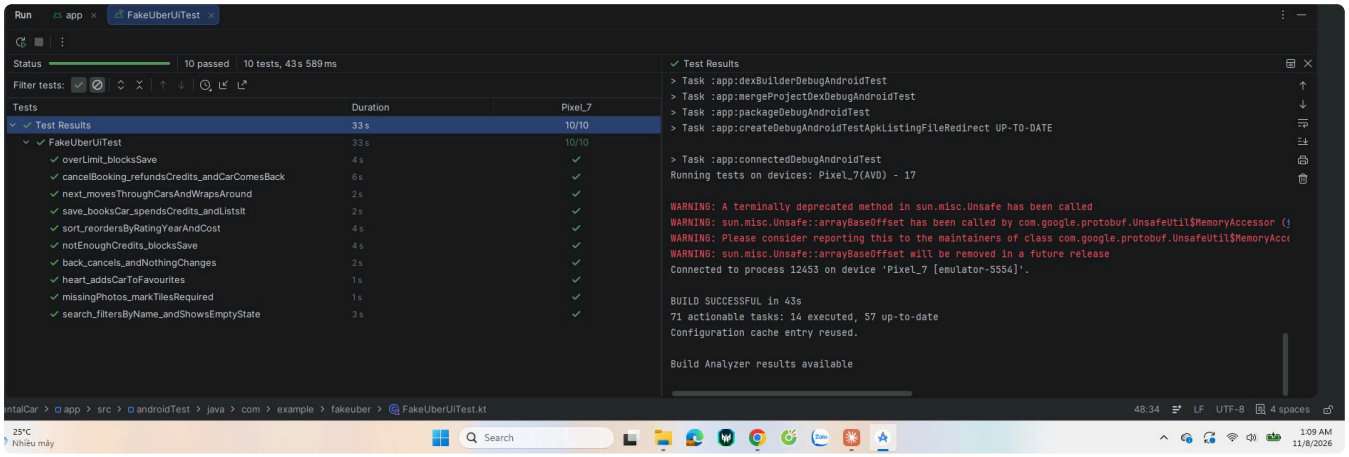


Figure 8. Android Studio test results: 10 passed in 43 s.

9. Challenges, explorations and takeaways

The hardest problems were not the features but Android's lifecycle: recreating Activities for dark mode caused three separate bugs, and UI testing exposed a real usability bug that manual testing had missed. I kept a development log throughout; these are the challenges that taught me the most.

Challenges

#	Problem	Cause	Fix
1	Balance, rentals and the chosen car reset when switching dark mode	`setDefaultNightMode()` destroys and recreates the Activity	All state moved into `MainViewModel` / `RentViewModel`; one `render()` redraws the screen
2	Switching to light mode on Rent made the app flicker between themes forever	On recreation Android restores each view's saved state after `onCreate()`; the stale Switch values on Home and Rent fired the listener and kept reversing each other	`saveEnabled="false"` on the Switch, and the listener ignores values equal to the current mode
3	After dark mode, the card jumped back to the first car while a search was active	The SearchView restores its text and fires `onQueryTextChange` again, which reset the index	`setQuery()` ignores unchanged text
4	Slider and screen could update each other in a loop	`render()` sets the slider value and the slider listener calls `render()`	The listener reacts only when the value differs from the ViewModel
5	Car photos used about 19 MB of RAM each	1024-px JPGs in `drawable/` are treated as mdpi and scaled 2.6×	Moved to `drawable-nodpi/`; picked photos are decoded at about 200 px
6	Slider track showed Material's default lavender	The theme did not define `colorSecondaryContainer`	Mapped every Material 3 colour role to the palette
7	Content slid under the status bar while scrolling	Edge-to-edge insets were applied as padding with `clipToPadding="false"`	`clipToPadding="true"` on Home

#	Problem	Cause	Fix
8	10 tests took 8 minutes	The Intent stub caught every outside Intent, including ActivityScenario's own helper, so each test waited for a 45-second timeout	Stub only the Photo Picker actions
9	The cancel-booking test failed	The "booked" Snackbar covered the Cancel booking button at the bottom of the page, so Espresso's tap hit the Snackbar; a real user had the same problem	120dp of bottom space on Home so the last button scrolls above the Snackbar; the test waits for the Snackbar to leave
10	Writing `Parcelable` correctly	Fields read in a different order from the write silently corrupt data	Wrote `writeToParcel` and the reading constructor side by side, field for field

Explorations beyond the weekly labs

- Activity Result API (`registerForActivityResult`) instead of the deprecated `startActivityForResult`
- The system Photo Picker, which needs no storage permission
- ViewModel for state that survives configuration changes
- Material 3 colour roles with `values-night` for a single-theme dark mode
- ViewBinding with `<include>` for reusable stat chips, spec cards and document tiles
- RecyclerView with an Adapter and ViewHolder for the Favourites row
- `espresso-intents`, custom ViewActions and the robot pattern

Takeaways

- Recreation is normal, not an edge case: keep state out of Activities and guard every listener that has side effects.
- Accessibility is cheapest when it is built into the colour palette and dimensions from the start.
- UI tests find real usability bugs, not just code bugs (challenge 9).
- Sketching the widgets before coding made the build mostly a matter of following the plan.

10. Time log

The project ran from 4 to 8 November 2026, with roughly the first half spent on planning, research and sketching and the second half on building and testing.

Date	Work
4 Nov 2026	Read the brief, made the plan (Google Sheet), first UI/UX research
5–6 Nov	Sketch iterations v4 to v8 (14 screens), Figma version, accessibility notes
7 Nov	Final UI/UX report (27 sources), UX design (user stories, use cases), market research for the 5 cars and free photos, low-fi Layouts A and B
7 Nov, evening	Created the Android Studio project; data model, Parcelable classes, in-memory repository, images; colours, theme and styles; Home screen and dark-mode switch
8 Nov, early morning	Rent screen with Intent, Parcelable, Activity Result API, validation and Photo Picker; favourites, search, sort, My rentals and cancel booking; manual test of every feature
8 Nov, early morning	10 Espresso tests: first run 9/10 in 8 min 5 s, then 10/10 in 43 s after fixing the stub and the Snackbar bug
8 Nov, morning	Fixed the dark-mode flicker on Rent; screenshots; this report

11. References, image credits and AI use

References

References follow APA 7th edition. Pages with no publication date are cited as n.d. with the date they were retrieved.

Android Developers. (n.d.-a). *Espresso-Intents*. Retrieved November 8, 2026, from https://developer.android.com/training/testing/espresso/intents

Android Developers. (n.d.-b). *Get a result from an activity*. Retrieved November 8, 2026, from https://developer.android.com/training/basics/intents/result

Android Developers. (n.d.-c). *Intents and intent filters*. Retrieved November 8, 2026, from https://developer.android.com/guide/components/intents-filters

Android Developers. (n.d.-d). *Parcelable*. Retrieved November 8, 2026, from https://developer.android.com/reference/android/os/Parcelable

Android Developers. (n.d.-e). *Photo picker*. Retrieved November 8, 2026, from https://developer.android.com/training/data-storage/shared/photopicker

Android Developers. (n.d.-f). *Toasts overview*. Retrieved November 8, 2026, from https://developer.android.com/guide/topics/ui/notifiers/toasts

Android Developers. (n.d.-g). *ViewModel overview*. Retrieved November 8, 2026, from https://developer.android.com/topic/libraries/architecture/viewmodel

Budiu, R. (2018). *Carousels on mobile devices*. Nielsen Norman Group. https://www.nngroup.com/articles/mobile-carousels/

Chen, D. (n.d.). *MAZDA3 SEDAN (BP) China* [Photograph]. Wikimedia Commons. https://commons.wikimedia.org/wiki/File:MAZDA3_SEDAN_(BP)_China.jpg

Febrian, A. (n.d.). *2021 Toyota Fortuner 2.4 TRD Sportivo (Indonesia) front view* [Photograph]. Wikimedia Commons. https://commons.wikimedia.org/wiki/File:2021_Toyota_Fortuner_2.4_TRD_Sportivo_(Indonesia)_front_view.jpg

JamesYoung8167. (2022). *Toyota Vios XP150 Facelift Shishi 01 2022-09-20* [Photograph]. Wikimedia Commons. https://commons.wikimedia.org/wiki/File:Toyota_Vios_XP150_Facelift_Shishi_01_2022-09-20.jpg

Llamas, E. (n.d.). *2025 VinFast VF 5 in Vinfast Blue, front left* [Photograph]. Wikimedia Commons. https://commons.wikimedia.org/wiki/File:2025_VinFast_VF_5_in_Vinfast_Blue,_front_left.jpg

Material Components for Android. (n.d.). *Snackbar* [Documentation]. GitHub. Retrieved November 8, 2026, from https://github.com/material-components/material-components-android/blob/master/docs/components/Snackbar.md

Neusesser, T., & Sunwall, E. (2023). *Error-message guidelines*. Nielsen Norman Group. https://www.nngroup.com/articles/error-message-guidelines/

Parhi, P., Karlson, A. K., & Bederson, B. B. (2006). Target size study for one-handed thumb use on small touchscreen devices. In *Proceedings of the 8th Conference on Human-Computer Interaction with Mobile Devices and Services* (pp. 203–210). Association for Computing Machinery. https://doi.org/10.1145/1152215.1152260

W3C. (2023). *Understanding success criterion 1.4.3: Contrast (minimum)*. https://www.w3.org/WAI/WCAG22/Understanding/contrast-minimum.html

オーバードライブ83. (2022). *2022 Mitsubishi Xpander Ultimate 1.5 NC2W (20220221)* [Photograph]. Wikimedia Commons. https://commons.wikimedia.org/wiki/File:2022_Mitsubishi_Xpander_Ultimate_1.5_NC2W_(20220221).jpg

Image credits

All car photos are from Wikimedia Commons under CC BY-SA 4.0, resized only.

Car	Commons file	Author
VinFast VF 5 Plus	2025 VinFast VF 5 in Vinfast Blue, front left	Ethan Llamas
Toyota Vios 1.5G	Toyota Vios XP150 Facelift Shishi 01 2022-09-20	JamesYoung8167
Mazda3 1.5	MAZDA3 SEDAN (BP) China	Dinkun Chen
Mitsubishi Xpander AT	2022 Mitsubishi Xpander Ultimate 1.5 NC2W (20220221)	オーバードライブ83
Toyota Fortuner 2.4AT	2021 Toyota Fortuner 2.4 TRD Sportivo (Indonesia) front view	Andra Febrian

Icons are Material Icons (Apache License 2.0).

Use of AI tools

I used Claude (Anthropic) as an assistant for UI/UX research, low-fi sketches, drafting code and drafting this report. I reviewed, built and ran all code myself in Android Studio, tested every feature by hand on the emulator, ran the Espresso suite, and found and reported the bugs described in section 9.